

Experiment 8
Interrupt handler

Name: Ashutosh Bajpayee
CDSID: abajpaye



Table of Contents

Sl. No.	Topic	Page No.
1	Introduction	3
2	Scope	3
3	Prerequisite	3
4	Connection Diagram	3
5	Working Principle	4
6	Application Code	4
7	Output	5
8	Hardware Image	5
9	Appendix	5

Introduction

An interrupt is a technique that enables a microcontroller to respond instantly to an external event, rather than continuously checking inputs through polling. In this experiment, pressing a push button generates an interrupt signal. The microcontroller detects this signal and executes an Interrupt Service Routine (ISR), which activates an LED blinking sequence. This method enhances system efficiency and responsiveness when compared to traditional polling-based approaches.

Scope

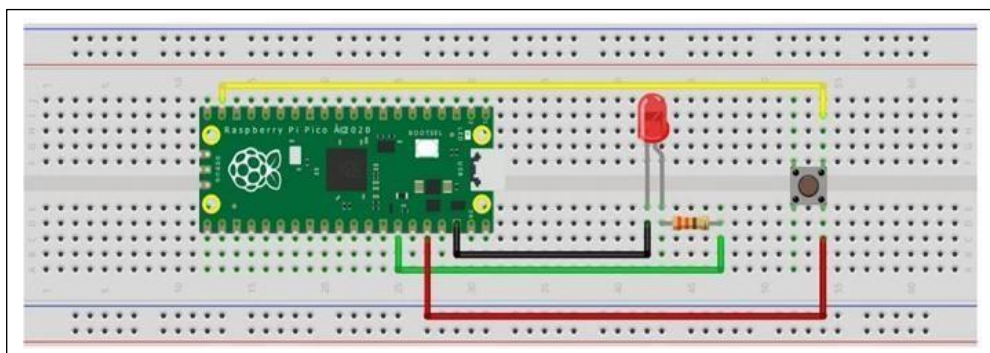
The scope of this experiment is to develop an understanding of interrupt handling in embedded systems and to interface a push button using GPIO interrupts. It also focuses on controlling an LED through an interrupt-driven approach and implementing real-time, event-driven programming using the Raspberry Pi Pico.

Prerequisite

Hardware: Raspberry Pi Pico Microcontroller, USB Connector, Breadboard, Resistor, Push Button, LED

Software: Visual studio code, Pico SDK, CMake & GCC Toolchain

Connection Diagram



Working Principle

The GPIO pin connected to the push button is configured as an input with a pull-down resistor, keeping it LOW by default. When the button is pressed, the signal changes from LOW to HIGH, creating a rising edge. This rising edge triggers a GPIO interrupt in the microcontroller. The interrupt handler (callback function) sets a flag, which is checked by the main loop. When the flag is detected, the LED blinks 5 times and then turns OFF, ensuring efficient operation without continuous polling.

Application code

```
#include <stdio.h>
#include "pico/stdlib.h"
#include "hardware/gpio.h"

#define LED_PIN 15
#define BUTTON_PIN 14

volatile bool interrupt_flag = false;

void gpio_callback(uint gpio, uint32_t events) {
    if (gpio == BUTTON_PIN) {
        interrupt_flag = true;
    }
}

int main() {
    stdio_init_all();

    sleep_ms(2000);

    printf("System Initialized...\n");

    gpio_init(LED_PIN);
    gpio_set_dir(LED_PIN, GPIO_OUT);

    gpio_init(BUTTON_PIN);
    gpio_set_dir(BUTTON_PIN, GPIO_IN);
    gpio_pull_down(BUTTON_PIN);

    gpio_set_irq_enabled_with_callback(
        BUTTON_PIN,
        GPIO_IRQ_EDGE_RISE,
        true,
        &gpio_callback
    );

    printf("Waiting for button press...\n");

    while (1) {
        if (interrupt_flag) {
            interrupt_flag = false;

            printf("Interrupt detected! Blinking LED...\n");

            for (int i = 0; i < 5; i++) {
                gpio_put(LED_PIN, 1);
                printf("LED ON (%d)\n", i+1);
                sleep_ms(200);

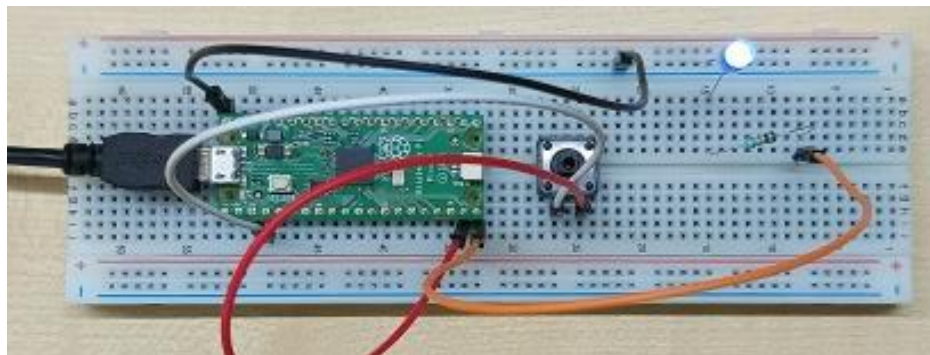
                gpio_put(LED_PIN, 0);
                printf("LED OFF (%d)\n", i+1);
                sleep_ms(200);
            }

            printf("Done blinking.\n");
        }
    }
}
```

Output

```
COM4 - PuTTY
Interrupt detected! Blinking LED...
LED ON (1)
LED OFF (1)
LED ON (2)
LED OFF (2)
LED ON (3)
LED OFF (3)
LED ON (4)
LED OFF (4)
LED ON (5)
LED OFF (5)
Done blinking.
Interrupt detected! Blinking LED...
LED ON (1)
LED OFF (1)
LED ON (2)
LED OFF (2)
LED ON (3)
LED OFF (3)
LED ON (4)
LED OFF (4)
LED ON (5)
LED OFF (5)
Done blinking.
```

Hardware Image



Appendix

- The experiment uses a Raspberry Pi Pico, push button, LED, and basic circuit components.
- GPIO functions are used for initialization, control, and interrupt handling through a callback function.
- When the button is pressed, a rising edge triggers an interrupt, and a flag is set.
- The main loop detects the flag and makes the LED blink five times, confirming correct operation.